Symbolic Recurrence Solving for Macro-Financial Models

Closed-form solutions for dynamic systems are useful for validating models, checking assumptions, and connecting symbolic derivations to downstream numeric workflows. This example features the `solveRecurrence` function, new in Symbolic Math Toolbox™ R2026a.

Workflow: symbolic first-principles -> closed-form recurrences -> sensitivity formulas -> Monte Carlo deployment

Requires: Symbolic Math Toolbox, Statistics and Machine Learning Toolbox™

1. Capital Accumulation: Single Linear Recurrence

The simplest dynamic model in macro-finance: capital in period $n+1$ is a fraction of last period's capital plus a constant injection.

$$k_{n+1} = \lambda k_n + c$$

where λ is the retention rate, such as $1-\delta$ for depreciation rate δ, and c is a constant investment flow.

Rather than simulate many periods, solve for the closed-form path k_n as a function of n, the initial capital k_0, and the parameters.

```
syms k(n) lambda c k0
assume(n, 'integer')
assume(n >= 0)
recurrence = k(n+1) == lambda*k(n) + c;
kSol = solveRecurrence(recurrence, k, k(0) == k0);
```

Closed-form capital path:

```
kSol
```

```
kSol =
```

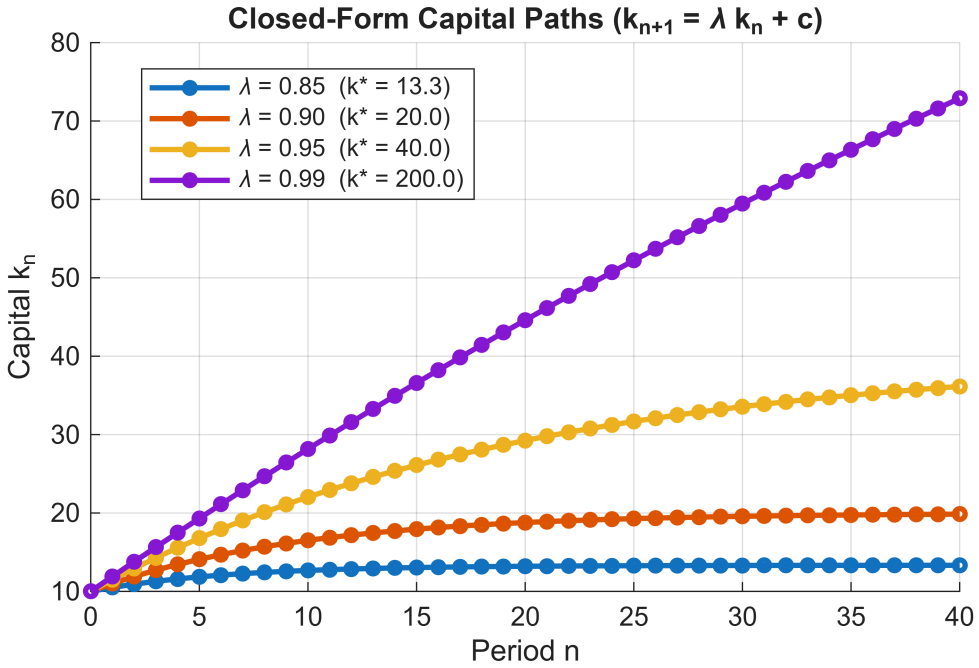
$$\lambda^n \left(k_0 + \frac{c}{\lambda - 1}\right) - \frac{c}{\lambda - 1}$$

Visualize Closed-Form Paths for Different Retention Rates

Evaluate the symbolic solution across a range of λ values.

```
figure('Name', 'Capital Accumulation Paths')
```

```
periods = 0:40;
lambda_vals = [0.85, 0.90, 0.95, 0.99];
c_val = 2;
k0_val = 10;
kFun = matlabFunction(kSol, 'Vars', {n, lambda, c, k0});
colors = lines(length(lambda_vals));
hold on
for idx = 1:length(lambda_vals)
    lam = lambda_vals(idx);
    kPath = kFun(periods, lam, c_val, k0_val);
    kss_simple = c_val/(1 - lam);
    plot(periods, kPath, '-o', 'Color', colors(idx,:), 'LineWidth', 2, ...
        'MarkerSize', 4, 'DisplayName', sprintf('\\lambda = %.2f  (k* = %.1f)', lam, kss_simple))
end
hold off
xlabel('Period n')
ylabel('Capital k_n')
title('Closed-Form Capital Paths (k_{n+1} = \lambda k_n + c)')
legend('Location', 'best')
grid on
```



2. Macro-Financial VAR: A 2x2 Recurrence

Reduced-form macro-financial workflows often propagate shocks through a linear state-space model. Here the state contains an output gap $\hat{y}_n$ and a credit spread q_n.

$$\hat{y}_{n+1} = \rho_y \hat{y}_n - \gamma_q q_n$$

$$q_{n+1} = -\gamma_y \hat{y}_n + \rho_q q_n$$

This is an ordinary recurrence for predetermined state variables, so initial-value shock propagation and eigenvalue stability checks are appropriate.

```
syms ygap(n) qspread(n) rho_y rho_q gamma_q gamma_y ygap0 q0
A_var = [rho_y,  -gamma_q;
         -gamma_y, rho_q];
```

Transition matrix:

```
A_var
```

A_var =

$$\begin{bmatrix} \rho_y & -\gamma_q \\ -\gamma_y & \rho_q \end{bmatrix}$$

```
eig_A_var = simplify(eig(A_var))
```

eig_A_var =

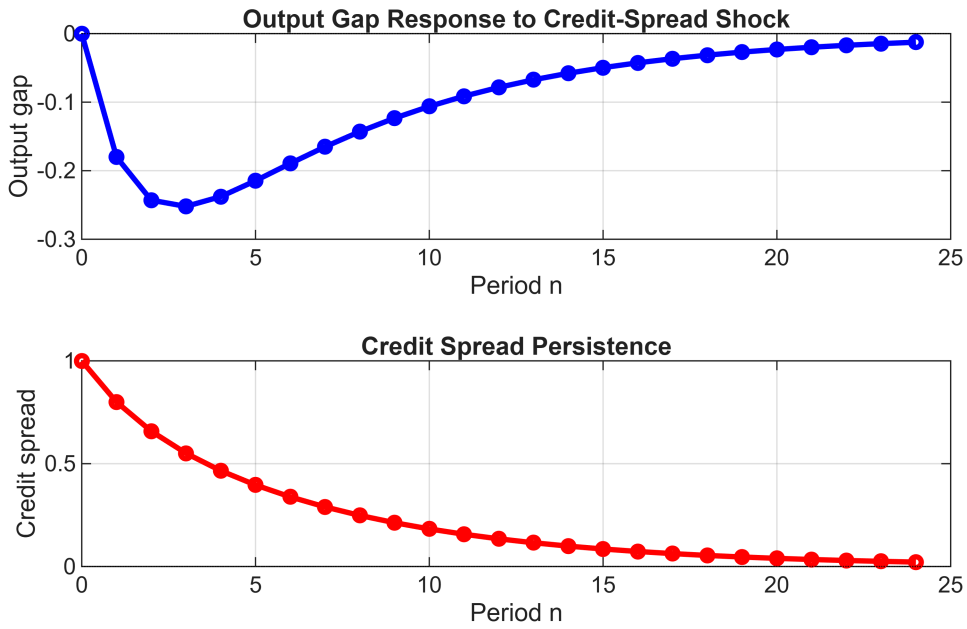
$$\begin{bmatrix} \frac{\rho_q}{2} + \frac{\rho_y}{2} - \frac{\sqrt{\rho_q^2 - 2\rho_q\rho_y + \rho_y^2 + 4\gamma_q\gamma_y}}{2} \\ \frac{\rho_q}{2} + \frac{\rho_y}{2} + \frac{\sqrt{\rho_q^2 - 2\rho_q\rho_y + \rho_y^2 + 4\gamma_q\gamma_y}}{2} \end{bmatrix}$$

Closed-Form Shock Propagation

`solveRecurrence` handles the 2x2 system directly. Calibrate the transition matrix and compute the response to a one percentage point credit-spread shock.

```
var_eqs = [ygap(n+1) == A_var(1,1)*ygap(n) + A_var(1,2)*qspread(n), ...
           qspread(n+1) == A_var(2,1)*ygap(n) + A_var(2,2)*qspread(n)];
var_init = [ygap(0) == ygap0, qspread(0) == q0];
sol_var = solveRecurrence(var_eqs, [ygap qspread], var_init);
ygap_sol = sol_var(1);
qspread_sol = sol_var(2);
cal_var = [rho_y, rho_q, gamma_q, gamma_y, ygap0, q0];
cal_var_vals = [0.55, 0.80, 0.18, 0.10, 0, 1];
ygap_irf = matlabFunction(simplify(subs(ygap_sol, cal_var, cal_var_vals)), 'Vars', {n});
qspread_irf = matlabFunction(simplify(subs(qspread_sol, cal_var, cal_var_vals)), 'Vars', {n});
periods_irf = 0:24;
figure('Name', 'Macro-Financial VAR Shock Propagation')
tiledlayout(2,1)
nexttile
plot(periods_irf, ygap_irf(periods_irf), 'b-o', 'LineWidth', 2, 'MarkerSize', 4)
yline(0, ':k')
xlabel('Period n')
```

```
ylabel('Output gap')
title('Output Gap Response to Credit-Spread Shock')
grid on
nexttile
plot(periods_irf, qspread_irf(periods_irf), 'r-o', 'LineWidth', 2, 'MarkerSize', 4)
yline(0, ':k')
xlabel('Period n')
ylabel('Credit spread')
title('Credit Spread Persistence')
grid on
```

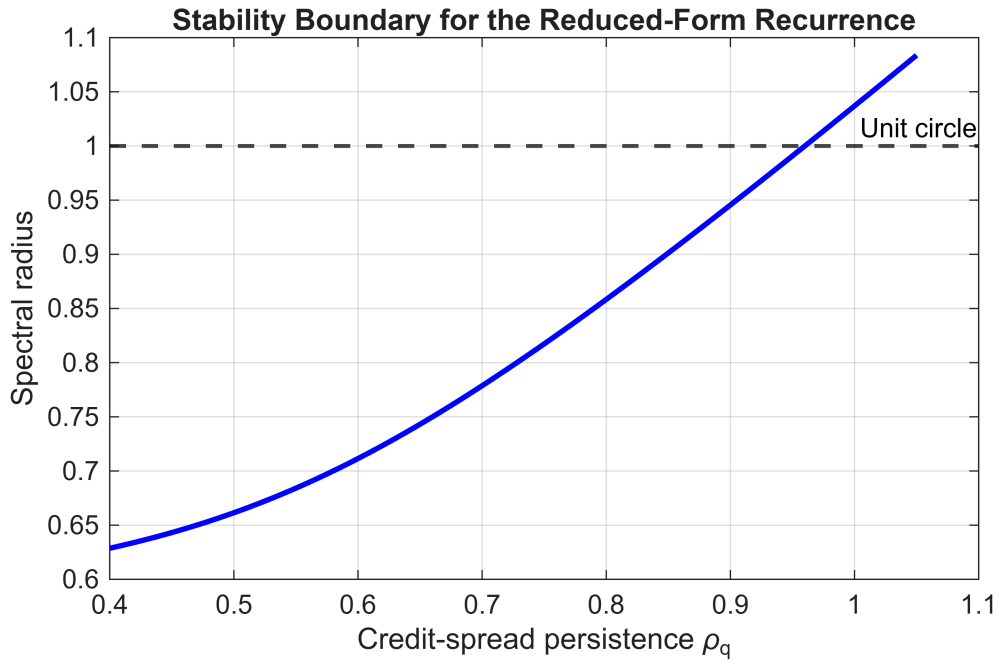


Stability Under Spread Persistence

The symbolic eigenvalues let us ask how close the recurrence gets to the unit circle as credit-spread persistence changes.

```
A_var_cal_partial = subs(A_var, [rho_y, gamma_q, gamma_y], [0.55, 0.18, 0.10]);
eig_var_partial = simplify(eig(A_var_cal_partial));
rho_q_range = linspace(0.40, 1.05, 200);
```

```
eig_var_fun = matlabFunction(eig_var_partial, 'Vars', {rho_q});
eig_vals = eig_var_fun(rho_q_range);
if size(eig_vals, 1) == 2
    spectral_radius = max(abs(eig_vals), [], 1);
else
    spectral_radius = max(abs(eig_vals), [], 2).';
end
figure('Name', 'VAR Spectral Radius vs Credit-Spread Persistence')
plot(rho_q_range, spectral_radius, 'b-', 'LineWidth', 2)
yline(1, '--k', 'Unit circle', 'LineWidth', 1.5)
xlabel('Credit-spread persistence \rho_q')
ylabel('Spectral radius')
title('Stability Boundary for the Reduced-Form Recurrence')
grid on
```



3. Solow Growth Model: Nonlinear Recurrence via Linearization

The neoclassical growth model features a nonlinear recurrence:

$$k_{n+1} = sk_n^{\alpha} + (1-\delta)k_n$$

This recurrence has no general closed-form solution. A standard symbolic workflow is to derive the steady state, linearize around it, and solve the local recurrence exactly.

```
syms s alpha delta k_init positive
assumeAlso(alpha < 1)
assumeAlso(delta < 1)
% Steady state: k* = s*k*^alpha + (1-delta)*k*
kss = (s/delta)^(1/(1 - alpha));
```

Steady-state capital:

```
kss
```

```
kss =
```

$$\frac{1}{\left(\frac{s}{\delta}\right)^{\frac{1}{\alpha-1}}}$$

```
syms k_sym positive
f_k = s*k_sym^alpha + (1 - delta)*k_sym;
lambda_star = simplify(subs(diff(f_k, k_sym), k_sym, kss));
```

Local convergence rate:

```
lambda_star
```

```
lambda_star =
```

$$\alpha\,\delta - \delta + 1$$

```
% Local deviation recurrence: x_{n+1} = lambda* x_n
syms x(n)
xSol = solveRecurrence(x(n+1) == lambda_star*x(n), x, x(0) == k_init - kss);
kSolowLinear = simplify(kss + xSol);
```

Linearized Solow capital path:

```
kSolowLinear
```

```
kSolowLinear =
```

$$\left(k_{\text{init}} - \frac{1}{\left(\frac{s}{\delta}\right)^{\frac{1}{\alpha-1}}}\right)(\alpha\,\delta - \delta + 1)^n + \frac{1}{\left(\frac{s}{\delta}\right)^{\frac{1}{\alpha-1}}}$$

Sensitivities of the Deployed Formula

Symbolic differentiation gives analytic sensitivities of terminal capital with respect to savings, capital share, and depreciation.

```
dkterminal_ds = simplify(diff(kSolowLinear, s));
dkterminal_dalpha = simplify(diff(kSolowLinear, alpha));
dkterminal_ddelta = simplify(diff(kSolowLinear, delta));
halflife = simplify(log(2)/log(1/lambda_star));
cal_solow = [s, alpha, delta, k_init, n];
cal_solow_vals = [0.2, 0.33, 0.05, 100, 20];
fprintf(['--- Linearized Solow model at s=0.2, alpha=0.33, delta=0.05 ---\n' ...
    'lambda*          = %.4f\n' ...
    'Half-life        = %.1f periods\n' ...
    'k_20 approx      = %.2f\n' ...
    'd(k_20)/ds       = %+.2f\n' ...
    'd(k_20)/dalpha   = %+.2f\n' ...
    'd(k_20)/ddelta   = %+.2f\n\n'], ...
    double(subs(lambda_star, cal_solow(1:3), cal_solow_vals(1:3))), ...
    double(subs(halflife, cal_solow(1:3), cal_solow_vals(1:3))), ...
    double(subs(kSolowLinear, cal_solow, cal_solow_vals)), ...
    double(subs(dkterminal_ds, cal_solow, cal_solow_vals)), ...
    double(subs(dkterminal_dalpha, cal_solow, cal_solow_vals)), ...
    double(subs(dkterminal_ddelta, cal_solow, cal_solow_vals)))
```

```
--- Linearized Solow model at s=0.2, alpha=0.33, delta=0.05 ---
lambda*          = 0.9665
Half-life        = 20.3 periods
k_20 approx      = 54.50
```

```
d(k_20)/ds       = +29.20
d(k_20)/dalpha   = +60.28
d(k_20)/ddelta   = -762.61
```

4. Downstream Deployment: Monte Carlo Stress Test

Convert the symbolic results to optimized MATLAB® functions and use them inside a Monte Carlo engine. The symbolic sensitivities carry through as analytic parameter derivatives, so no bump-and-revalue runs are needed.

Generate Deployable Functions

```
matlabFunction(kSol, 'File', 'capitalPath', 'Vars', {n, lambda, c, k0}, ...
    'Outputs', {'k_n'});
matlabFunction(kSolowLinear, 'File', 'solowTerminalCapital', ...
    'Vars', {n, k_init, s, alpha, delta}, 'Outputs', {'k_terminal'});
matlabFunction(kSolowLinear, dkterminal_ds, dkterminal_dalpha, dkterminal_ddelta, ...
    'File', 'solowTerminalSensitivity', 'Vars', {n, k_init, s, alpha, delta}, ...
    'Outputs', {'k_terminal', 'dkterminal_ds', 'dkterminal_dalpha', 'dkterminal_ddelta'});
fprintf(['Generated: capitalPath.m, solowTerminalCapital.m, solowTerminalSensitivity.m\n' ...
    '  (also available via ccode() for C/C++ integration)\n\n'])
```

```
Generated: capitalPath.m, solowTerminalCapital.m, solowTerminalSensitivity.m
  (also available via ccode() for C/C++ integration)
```

Monte Carlo: Parameter Uncertainty -> Capital Distribution

A risk team wants to know: given uncertainty in economic parameters, what is the distribution of capital after N periods?

Draw random parameter scenarios, evaluate the linearized Solow path in one vectorized call, and compute analytic sensitivities at each draw.

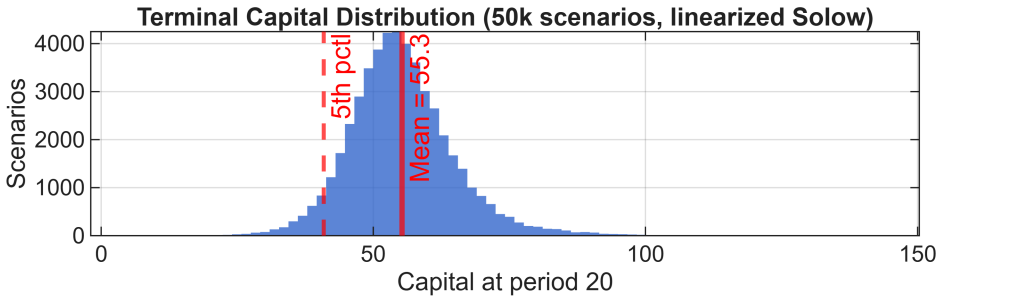
```
N_periods = 20;
N_scenarios = 50000;
k0_val = 100;
% Calibrated parameter distributions (Statistics and Machine Learning Toolbox)
% Savings rate: truncated normal, bounded to (0, 1) as required economically
pd_s = makedist('Normal', 'mu', 0.20, 'sigma', 0.03);
pd_s = truncate(pd_s, 0.01, 0.99);
% Capital share: truncated normal, bounded to (0, 1)
```

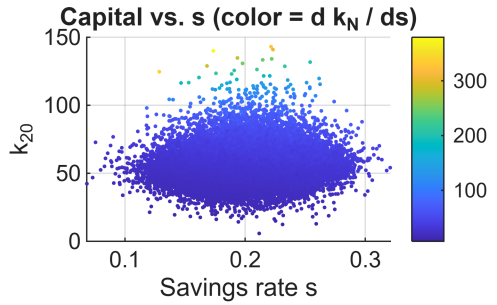
```
pd_alpha = makedist('Normal', 'mu', 0.33, 'sigma', 0.02);
pd_alpha = truncate(pd_alpha, 0.01, 0.99);
% Depreciation shock: fat-tailed t-distribution captures regime stress
pd_delta = makedist('tLocationScale', 'mu', 0.05, 'sigma', 0.01, 'nu', 5);
pd_delta = truncate(pd_delta, 0.001, 0.5);
rng(42)
s_draws = random(pd_s, N_scenarios, 1);
alpha_draws = random(pd_alpha, N_scenarios, 1);
delta_draws = random(pd_delta, N_scenarios, 1);
% Linearized Solow terminal capital: one vectorized call, no period loop
k_terminal = solowTerminalCapital(N_periods, k0_val, s_draws, alpha_draws, delta_draws);
% Analytic terminal-capital sensitivities at each scenario
[~, dkds, ~, dkdd] = solowTerminalSensitivity(N_periods, k0_val, s_draws, alpha_draws, delta_draws);
```

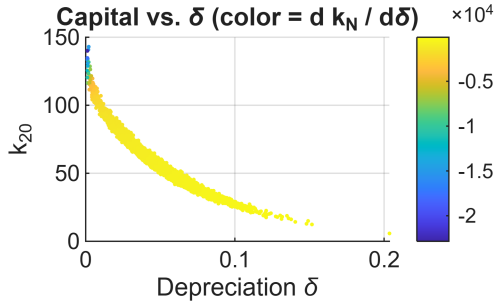
Results

```
figure('Name', 'Monte Carlo - Linearized Solow Terminal Capital')
tiledlayout(2,2)
nexttile([1 2])
histogram(k_terminal, 80, 'FaceColor', [0.2 0.4 0.8], 'EdgeColor', 'none', ...
    'FaceAlpha', 0.8)
xline(mean(k_terminal), 'r-', sprintf('Mean = %.1f', mean(k_terminal)), 'LineWidth', 2)
xline(quantile(k_terminal, 0.05), 'r--', '5th pctl', 'LineWidth', 1.5)
xlabel(sprintf('Capital at period %d', N_periods))
ylabel('Scenarios')
title(sprintf('Terminal Capital Distribution (%dk scenarios, linearized Solow)', N_scenarios/1000))
grid on
nexttile
scatter(s_draws, k_terminal, 2, dkds, 'filled')
colorbar
xlabel('Savings rate s')
ylabel(sprintf('k_{%d}', N_periods))
title('Capital vs. s (color = d k_N / ds)')
grid on
nexttile
scatter(delta_draws, k_terminal, 2, dkdd, 'filled')
colorbar
```

```
xlabel('Depreciation \delta')
ylabel(sprintf('k_{%d}', N_periods))
title('Capital vs. \delta (color = d k_N / d\delta)')
grid on
```







Computational Note

The Monte Carlo calculation deploys the linearized Solow formula derived symbolically in Section 3. It is a local closed-form approximation around the steady state, not an exact solution of the nonlinear Solow recurrence for large deviations.

The terminal-capital sensitivities are evaluated directly from symbolic derivative formulas in `solowTerminalSensitivity`. This gives analytic sensitivities for each scenario without finite-difference bump-and-revalue runs.